Rapport

1. Choix du langage

Langage : C.

Justification :

- C'est le langage du cours (structures de données, tris, stratégies algorithmiques). Le projet reste dans le même cadre.
- Il oblige à gérer la mémoire soi-même (malloc, realloc, free). On voit donc le coût réel des structures, en temps et en espace.

2. Organisation du code

Fichier	Rôle
src/colis.h, src/colis.c	Structure Colis, collection, ajout, affichage, recherche par id
src/tri.h, src/tri.c	Tri fusion et affichages triés (poids, valeur, priorité)
src/teste1.c, src/teste2.c	Programmes de test (ajout et recherche ; tris)
src/camion.h, src/camion.c, src/teste3.c, src/teste4.c	Chargement du camion : sac à dos par programmation dynamique, glouton de comparaison, version avec priorités
src/bench.c	Mesures de performance (100, 1 000, 10 000, 100 000 colis)
src/main.c	Programme principal : menu qui relie toutes les fonctionnalités
Makefile, README.md	Compilation, mode d'emploi

3. Gestion des colis

Choix 1 : Stockage des colis (tableau dynamique)

Élément	Contenu
Problème	Stocker un nombre de colis inconnu à l'avance (de 100 à 100 000 et plus), avec des ajouts fréquents.

Élément	Contenu
Solution	Un tableau dynamique de Colis. Quand il est plein, sa capacité double (realloc). Capacité initiale : 16.
Temps	Ajout : O(1) amorti (une copie O(n) seulement à chaque doublement, soit n copies en tout pour n ajouts). Accès par index : O(1). Affichage : O(n).
Espace	O(n). Au plus 2n cases réservées, car on double à chaque fois. Un colis occupe 48 octets (4 entiers et 32 caractères de destination).
Alternative	La liste chaînée : ajout en O(1) aussi, mais pas d'accès par index et mémoire dispersée. Elle gêne les tris et ralentit les parcours. Un tableau de taille fixe : plus simple, mais impossible à agrandir.
Justification	La mémoire est contiguë : les parcours et les tris sont rapides. On accède directement au i-ème colis, ce qui sert au tri et au sac à dos. Le doublement évite de recopier à chaque ajout (ajouter 1 case à la fois serait O(n) par ajout, donc $O(n^2)$ au total).

Choix 2 : Recherche d'un colis par identifiant (table de hachage)

Élément	Contenu
Problème	Retrouver un colis à partir de son id, et refuser les doublons à l'ajout. L'opération est répétée très souvent (section 6 de l'énoncé).
Solution	Une table de hachage avec résolution des collisions par chaînage (separate chaining). Fonction de hachage multiplicative de Knuth (id × 2654435761 mod taille). La table double quand le facteur de charge dépasse 0,75. Elle stocke l'index du colis dans le tableau, et non le colis lui-même.
Temps	Recherche et ajout : O(1) en moyenne. Pire cas O(n) (tous les ids dans la même chaîne). Agrandissement de la table : O(n), mais rare, donc O(1) amorti.
Espace	O(n) en plus du tableau : un maillon de 16 octets par colis, et un pointeur de 8 octets par case de la table.
Alternative	Recherche linéaire : O(n), sans mémoire en plus. Elle est conservée dans le code (collection_chercher_lineaire) comme référence de mesure. Tableau trié par id + recherche dichotomique : O(log n), mais chaque insertion coûte O(n) pour garder l'ordre. Arbre équilibré : O(log n) partout, mais plus complexe à écrire.
Justification	Les ajouts et les recherches sont fréquents : le hachage donne les deux en O(1). La dichotomique serait pénalisée par les insertions, et la linéaire devient trop lente à 100 000 colis. On garde un facteur de

Élément	Contenu
	charge ≤ 0,75 pour que les chaînes restent courtes (longueur moyenne inférieure à 1). Stocker l'index évite qu'un realloc du tableau invalide les pointeurs.

Choix 3 : Affichage trié (tri fusion)

Élément	Contenu
Problème	Afficher les colis triés par poids, par valeur ou par priorité, jusqu'à 100 000 colis, sans modifier la collection.
Solution	Tri fusion (*Merge Sort*), écrit à la main : on coupe en deux, on trie chaque moitié, on fusionne. On trie un tableau de pointeurs vers les colis, avec un comparateur différent selon le critère. Ordre croissant ou décroissant au choix.
Temps	O(n log n) dans tous les cas (meilleur, moyen et pire).
Espace	O(n) : un tableau de pointeurs trié, plus un tableau temporaire de fusion (8 octets par colis chacun).
Alternative	Quick Sort : souvent plus rapide en pratique et en O(1) de mémoire en plus, mais $O(n^2)$ avec un mauvais pivot, et non stable. Tris simples (insertion, sélection, bulles) : $O(n^2)$, inutilisables à 100 000 colis (environ 10^{10} opérations). Tri par comptage pour la priorité (5 valeurs seulement) : O(n), mais spécifique à ce critère.
Justification	Le temps est garanti quelle que soit la distribution des données. Le tri est stable : à valeur égale, les colis restent dans l'ordre d'ajout, ce qui compte beaucoup pour la priorité (seulement 5 valeurs, donc beaucoup d'égalités). Le surcoût mémoire est acceptable. Trier des pointeurs plutôt que des colis de 48 octets rend les déplacements moins coûteux et laisse la collection d'origine intacte.

4. Préparation du camion

Choix 4 : Sélection des colis du camion (programmation dynamique)

Élément	Contenu
Problème	Choisir un ensemble de colis dont le poids total est ≤ capacité W, avec la plus grande valeur totale possible. C'est le problème du sac à dos 0/1 : chaque colis est pris en entier ou pas du tout.
Solution	Programmation dynamique. dp[w] = meilleure valeur avec un poids ≤ w. Pour chaque colis (poids p, valeur v) et pour w de W à p (en décroissant) : dp[w] = max(dp[w], dp[w-p] + v). Le parcours décroissant garantit qu'un colis n'est utilisé qu'une fois. Pour savoir quels colis sont pris, on garde un bit keep(i, w) quand prendre le colis i améliore dp[w], puis on remonte du dernier colis au premier. La capacité est ramenée à la somme des poids si elle est plus grande (inutile de la dépasser).
Temps	O(n × W) (n colis, W capacité). C'est un temps pseudo-polynomial : il dépend de la valeur de W, pas seulement de sa taille. La reconstruction coûte O(n).
Espace	O(W) pour dp (8 octets par case), plus n × W bits pour la reconstruction. Exemple : 100 000 colis et W = 500 donnent 6,25 Mo. Une table complète d'entiers n × W en demanderait 32 à 64 fois plus. Si n × W bits dépasse 512 Mo, le programme refuse proprement (code d'erreur) au lieu de planter.
Alternative	Glouton (trier par valeur/poids décroissant, prendre tant que ça rentre) : O(n log n), très rapide, mais pas optimal. Recherche exhaustive / backtracking : essayer les 2^n sous-ensembles, optimal mais inutilisable au-delà d'environ 30 colis. Table n × W complète : plus simple à écrire, mais beaucoup plus de mémoire.
Justification	La solution optimale du problème se construit à partir des solutions optimales des sous-problèmes (principe d'optimalité) et les sous-problèmes se recoupent : c'est le cas typique de la programmation dynamique, qui évite les calculs répétés de la recherche exhaustive. Le résultat est toujours optimal, ce que ne garantit pas le glouton.

La solution est-elle toujours optimale ? Oui : la récurrence considère, pour chaque colis et chaque capacité, les deux seules possibilités (prendre / ne pas prendre) et garde la meilleure. Vérifié par programme : sur 300 jeux de données aléatoires (0 à 12 colis), le résultat est égal à celui de la force brute (2^n sous-ensembles), et le poids ne dépasse jamais la capacité.

Le glouton n'est pas optimal. Contre-exemple : colis (poids 10, valeur 60), (20, 100), (30, 120), capacité 50. Le glouton prend les deux premiers (valeur 160), alors que le meilleur choix est les deux derniers (valeur 220). Sur les 300 tests aléatoires, le glouton a été sous-optimal 42 fois (14 %). Son seul intérêt est la vitesse.

Mesure : 100 000 colis et capacité 500 kg : environ 40 ms.

Limite : si les poids sont exprimés dans une petite unité (grammes, W de l'ordre de 10^6 ou plus), O(n × W) et la mémoire deviennent importants. Dans ce cas, on peut convertir les poids en une unité plus grosse (kg), ou passer à un algorithme approché.

5. Prise en compte des priorités

Choix 5 : Règle de score valeur × priorité

Élément	Contenu
Problème	Favoriser les colis de priorité élevée dans le chargement, sans formule imposée, en tenant compte du poids, de la valeur et de la priorité.
Solution	On remplace la valeur de chaque colis par un score : score = valeur × (base + priorité − 1), avec base = 4 par défaut. Les coefficients valent 4, 5, 6, 7, 8 pour les priorités 1 à 5 : à valeur égale, un colis de priorité 5 compte 2 fois plus qu'un colis de priorité 1. On relance ensuite le même sac à dos par programmation dynamique, mais en maximisant la somme des scores. Le poids reste la contrainte (poids total ≤ capacité). Le paramètre base règle l'effet : plus il est petit, plus la priorité pèse (base 1 : ×5) ; plus il est grand, moins elle pèse (base 1000 : l'effet disparaît). Il doit être compris entre 1 et 1000.
Temps	O(n × W), comme le sac à dos simple. Le calcul des scores coûte O(n) en plus.
Espace	O(W + n × W / 8 octets), plus O(n) pour le tableau des scores (8 octets par colis).
Alternative	Ordre lexicographique (on charge d'abord tous les colis de priorité 5, puis ceux de priorité 4, etc.) : garantit la priorité, mais peut gaspiller la capacité et ignorer un colis de priorité 4 très rentable pour une petite place. Priorité « dure » (les colis de priorité 5 sont obligatoires) : utile si la règle métier l'impose, mais peut rendre le problème impossible si les colis obligatoires dépassent la capacité. Coefficient exponentiel (2^priorité) : la priorité devient presque absolue.
Justification	La règle est simple à expliquer et réglable par un seul paramètre. Elle reste linéaire en la valeur, donc la programmation dynamique garde sa garantie : le chargement est optimal pour le score choisi. Elle fait un compromis : la priorité favorise un colis sans interdire de charger un colis très rentable de priorité faible. Le poids intervient comme avant, par la contrainte de capacité, ce qui évite de charger un colis très lourd juste parce qu'il est prioritaire. Les coefficients sont entiers, donc aucun arrondi flottant ne fausse la comparaison.

Exemple chiffré:

Colis	Poids	Valeur	Priorité	Coefficient	Score
A	8	20	5	8	160
B	6	25	1	4	100
C	5	18	4	7	126
D	4	14	3	6	84

- **Sans priorité** : on charge B + D (poids 10, valeur 39), de priorité moyenne 2,00.
- **Avec priorité** (base 4) : on charge C + D (poids 9, score 210, valeur 32), de priorité moyenne 3,50. B, de priorité 1, est remplacé par C, de priorité 4.
- On accepte donc de perdre 7 de valeur (−18 %) pour embarquer des colis plus prioritaires. A (priorité 5) n'est pas chargé : il pèse 8 kg pour 160 de score, alors que C + D rapportent 210 pour 9 kg.
- Effet du paramètre : bases 1, 2 et 4 donnent C + D ; bases 10 et 1000 donnent B + D (la priorité n'influence plus la sélection).

Vérification : sur 300 jeux aléatoires, le résultat est égal à la force brute pour le score, la capacité est respectée, et la valeur pure ne dépasse jamais l'optimum sans priorité. En moyenne, la priorité moyenne passe de 2,96 à 3,11 pour une perte de valeur de 1,4 %. L'effet est faible sur ces données aléatoires, car la capacité y est souvent peu serrée ; il est net quand la place manque, comme dans l'exemple.

Limite : le score ne garantit pas qu'un colis de priorité 5 sera chargé (cas de A ci-dessus). Si l'entreprise veut une garantie, il faut une règle « dure » ou un coefficient plus fort (base 1).

6. Recherche et performance

Protocole de mesure

- Programme : src/bench.c (make bench puis ./bench). Compilation avec gcc -O2.
- Chronomètre : clock(). Chaque mesure est répétée plusieurs fois et on garde le meilleur temps (le moins perturbé par le système). Pour les opérations très courtes (recherche), on mesure un grand nombre d'appels et on divise.

- Données : n = 100, 1 000, 10 000 et 100 000 colis, générés avec une graine fixe (résultats reproductibles). Poids de 1 à 50, valeur de 0 à 999, priorité de 1 à 5.
- Les temps absolus dépendent de la machine ; ce sont les rapports entre tailles qui comptent.

Résultats

Ajout de n colis

n	temps total (ms)	par ajout (ns)
100	0,018	180
1 000	0,100	100
10 000	1,093	109
100 000	31,454	315

Recherche d'un colis par id

n	hachage (ns)	linéaire (ns)	linéaire / hachage
100	3	32	×11
1 000	3	172	×53
10 000	5	2 432	×505
100 000	21	64 502	×3 138

Tri par poids

n	tri fusion (ms)	tri par insertion (ms)	insertion / fusion
100	0,002	0,001	×0,5
1 000	0,045	0,098	×2
10 000	0,829	18,724	×23
100 000	11,308	4 315,959	×382

Sac à dos par programmation dynamique

Évolution avec n (capacité fixe W = 500 kg) :

n	temps (ms)
100	0,05
1 000	0,36
10 000	3,23
100 000	32,95

Évolution avec la capacité W (n = 10 000 colis) :

W	temps (ms)	table de reconstruction (Mo)
100	0,60	0,13
1 000	6,74	1,25
10 000	82,75	12,50
100 000	1 199,20	125,00

Opération 1 : recherche d'un colis par id

Élément	Analyse
Complexité	Hachage : O(1) en moyenne (O(n) au pire). Recherche linéaire : O(n).
Évolution quand n augmente	La recherche linéaire suit n : de 100 à 100 000 colis (n × 1 000), son temps est multiplié par environ 2 000 (32 ns → 64 502 ns). Le hachage reste quasi constant (3 ns → 21 ns). À 100 000 colis, le hachage est environ 3 000 fois plus rapide.
Le choix reste-t-il adapté ?	Oui. Le hachage est le bon choix : la recherche est fréquente et les données croissent. La recherche linéaire n'est acceptable que pour quelques centaines de colis.
Remarque	La légère hausse du hachage (3 → 21 ns) n'est pas algorithmique : à 100 000 colis, les données ne tiennent plus dans le cache du processeur et chaque accès coûte plus cher. La complexité reste O(1).
Amélioration possible	Si l'on cherche aussi par poids, valeur ou destination, il faut d'autres index : une seconde table de hachage (destination), ou un tableau trié avec recherche dichotomique O(log n) pour les

Élément	Analyse
	recherches par intervalle ("colis de 10 à 20 kg"). Le hachage ne sait pas faire de recherche par intervalle.

Opération 2 : tri des colis (affichage par poids, valeur, priorité)

Élément	Analyse
Complexité	Tri fusion : O(n log n). Tri par insertion (référence) : O(n²) au pire et en moyenne.
Évolution quand n augmente	Le tri fusion passe de 0,045 ms (n = 1 000) à 11,3 ms (n = 100 000) : environ ×250 pour n × 100, ce qui correspond à n log n (théorie : environ ×170, le reste vient du cache). Le tri par insertion passe de 0,098 ms à 4 316 ms : environ ×44 000 pour n × 100, ce qui correspond à n² (×10 000, plus les effets du cache). À 100 000 colis, il est 382 fois plus lent. À 100 colis, l'insertion est aussi rapide, voire un peu plus.
Le choix reste-t-il adapté ?	Oui. 11 ms pour 100 000 colis, c'est instantané. Le tri par insertion serait inutilisable (4 secondes). Il reste intéressant sur des données presque triées (O(n) dans ce cas) ou de très petite taille.
Amélioration possible	Pour la priorité, qui n'a que 5 valeurs, un tri par comptage est en O(n) et stable. Si l'on affiche souvent le même tri sans ajouter de colis, on peut garder le résultat en mémoire et ne le recalculer qu'après un ajout (cache invalidé à chaque ajout). Si les ajouts sont très fréquents, un tas ou un arbre équilibré maintient l'ordre en O(log n) par ajout.

Opération 3 : sélection du camion (sac à dos par programmation dynamique)

Élément	Analyse
Complexité	O(n × W) en temps ; O(W + n × W / 8 octets) en espace.
Évolution	Linéaire en n : ×10 colis donne environ ×10 de temps (0,36 → 3,23 → 32,95 ms). Linéaire en W : ×10 de capacité donne

Élément	Analyse
	environ ×10 à ×14 de temps et de mémoire (0,60 → 6,74 → 82,75 → 1 199 ms). Il faut donc surveiller W autant que n.
Le choix reste-t-il adapté ?	Oui pour des capacités de camion raisonnables (quelques milliers de kg) : 33 ms pour 100 000 colis et 500 kg. Il devient lourd si W atteint 10^5 ou plus (1,2 s et 125 Mo pour 10 000 colis).
Amélioration possible	Exprimer les poids dans une unité plus grande (kg plutôt que g) pour réduire W ; pré-filtrer les colis (retirer les colis plus lourds que W ou de valeur nulle) ; pour de très grandes capacités, utiliser un algorithme approché (glouton, ou schéma d'approximation) en acceptant une perte de valeur ; pour des valeurs plutôt petites que des poids, faire la DP sur la valeur (dimension = somme des valeurs) au lieu du poids. Le programme refuse proprement les cas qui dépasseraient 512 Mo de table.

Ajout de colis

L'ajout est O(1) amorti (doublement du tableau et de la table de hachage). Mesuré : environ 100 ns par ajout jusqu'à 10 000 colis, puis 315 ns à 100 000. La hausse vient des copies lors des agrandissements (rares mais de plus en plus grosses) et des accès hors cache. Le coût par ajout reste de l'ordre de la fraction de microseconde : l'ajout n'est pas un goulot d'étranglement.

Mémoire

Un colis occupe 48 octets, plus 16 octets de maillon de hachage et 8 à 16 octets de table de hachage (la capacité du tableau et la table sont des puissances de 2) : environ 70 à 100 octets par colis, soit environ 10 Mo pour 100 000 colis. La mémoire est O(n).

7. Programme final

Le programme livraisons (compilé avec make) propose un menu qui relie toutes les fonctionnalités :

Option du menu	**Fonction**	**Algorithme**
1. Afficher les colis	Liste des colis (limitée à 30 lignes par défaut si la liste est longue)	Parcours du tableau, O(n)
2. Ajouter un colis	Saisie contrôlée ; refus des doublons et des valeurs invalides	Tableau dynamique + hachage, O(1) amorti

Option du menu	Fonction	Algorithme
3. Rechercher un colis	Recherche par identifiant	Table de hachage, O(1) en moyenne
4. Organiser les colis	Tri par poids, valeur ou priorité, croissant ou décroissant	Tri fusion, O(n log n)
5. Préparer un camion	Valeur maximale, avec priorités, glouton, ou comparaison des trois	Programmation dynamique O(n × W) / glouton O(n log n)
6. Afficher le chargement	Colis chargés, répartition par destination, taux de remplissage	Parcours du chargement
7. Afficher les statistiques	Totaux, moyennes, répartition par priorité et par destination, état de la table de hachage, mémoire estimée	Parcours, O(n)
8. Jeu de test / import / vider	Les 5 colis de l'énoncé, N colis aléatoires (jusqu'à 1 000 000), import d'un fichier CSV	O(n)

Les saisies sont contrôlées (texte à la place d'un nombre, valeur hors limites) et le programme s'arrête proprement si l'entrée est fermée.

8. Tests et vérifications

- teste1 : ajout, doublons, valeurs invalides, recherche (hachage et linéaire concordent sur 20 000 colis).
- teste2 : les 6 tris (3 critères × 2 sens) sont corrects et stables sur 100 000 colis ; cas d'une collection vide.
- teste3 : la programmation dynamique égale la force brute sur 300 jeux aléatoires ; contre-exemple montrant que le glouton n'est pas optimal ; cas limites (capacité 0 ou négative, collection vide).
- teste4 : le résultat est optimal pour le score (comparé à la force brute) et respecte la capacité.
- Tous les programmes (make tests) passent, et valgrind ne détecte ni fuite mémoire ni accès invalide.